Assignment 4

1] Why we can't create obj of class which contains pure virtual function in it?

⇒ Ans 1: The vtable of that class is incomplete, as vtable is part of obj, so our obj is also incomplete.
* An object has to be complete.

2] Why do you mean by a Pure virtual function?

⇒ A. pure virtual () means function w/o body
ie. Abstract ()

Code: * purevirtual.cpp

ie. Parent Class - Abstract class.

3] What will happen if base class contains a virtual () under private access specifier.

⇒ 1) Derived class cannot override it directly.
2) Base class can provide a public / protected interface to access private virtual ()

Eg: class Base {
private:
virtual void show() = 0;
public:
void display() { show(); } // indirect access
};

4] What does it mean by a Concrete method & an Abstract method?

⇒ Concrete method - function with a body
Abstract method - (pure virtual ()) is a func w/o an body in base class

5] Explain given syntax & diagrammatic representation of Polymorphism with virtual functions & inheritance.

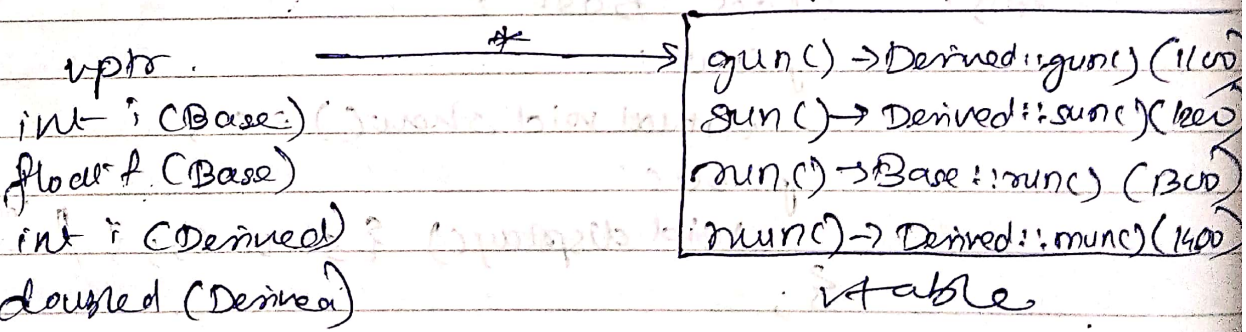
- Derived class overrides base class functions

1) Base class – 2 virtual() → gun() & sun()
   Abstract class ← can't create obj.
   virtual() run() – with implementation.

2) Derived class
   - Inherits base.
   - Implements gun() & sun() ← adds own function
     fun() → not virtual
   - Defines → virtual() - run()

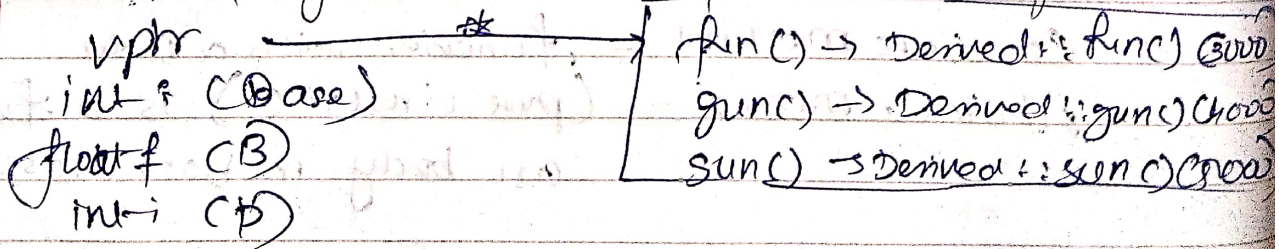
- Base class pointer (Base* bp) is used to store address of a derived object

1) bp → fun() – Error, as fun() is not virtual
2) bp → gun() – Resolved via vtable → Calls Derived::gun()
3) bp → sun() – Calls Derived::sun()
4) bp → run() – Base::run(), as not overridden
5) bp → mun() – Calls Derived::mun()



6] Memory layout & Assembly instructions of function

→

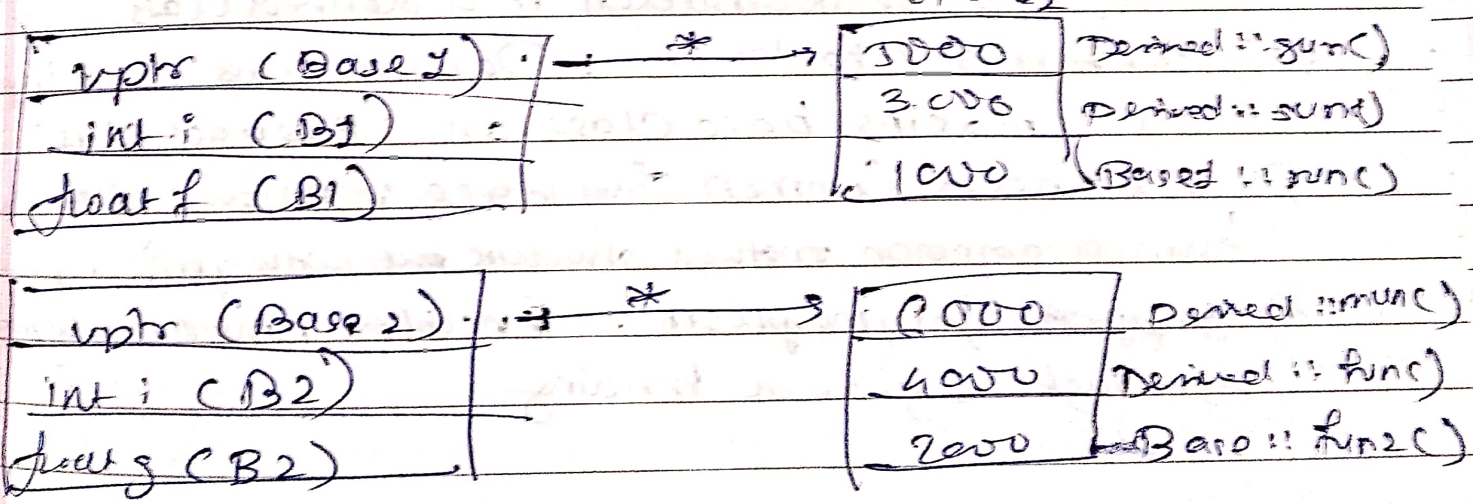


Assembly instructions

1] mov RAX [bp] - Load address of obj (Derived) in RAX
2] — " — [RAX] - Load vtable pointer from obj
3] — " — [RAX + 8] - Load address of Derived::fun()
from vtable
5] CALL RDX - Call Derived::fun()

Derived::fun()
1] PUSH RBP - Save base pointer
2] mov RBP, RSP - Set up new base pointer
3] SUB RSP, 16 ; Allocate stack space
4] LEA RDI, "Derived fun" - Load string address
5] CALL printf - Print "Derived fun"
6] ADD RSP, 16 - Restore stack
7] POP RBP - Restore old base pointer
8] RET - Return to caller.

Draw object layout of below syntax.



8] Different ways to achieve UPCASTING in OOPL:

⇒ Upcasting - Pointer having less capacity, points to large data capacity pointer

1] Implicit Upcasting (Automatic):

1) When a derived class object is assigned to a base class pointer or reference

2] Explicit upcasting using Typecasting:

- When derived class pointer is explicitly cast to a base class pointer

3] using Static Cast:

- Ensures compile-time safety.

4] Dynamic Cast:

- used in Run-time Type Identification (RTTI) when base class has least one virtual function.

Code of virtual.cpp

9] Pure virtual Function:

- A func declared in base class but has no definition & must be overridden in a derived class

Uses: 1) Enforces Inheritance :- Derived class must implement (=0), making base class an 'Abstract Class'

2) Defines a Common Interface: Allows diff classes to have a common method structure but with specific implementation

3) Supports Polymorphism: Enables runtime polymorphism through Dynamic binding

10] Can we create a Pointer of a Class that contains a (pure virtual()?

→ Yes we can, but we can't instantiate an object of that class

Eg: class Base {

public:

virtual void show() = 0; } // Pure virtual()

class Derived : public Base {
public:
    void show() override { cout << "Der Class" << endl; }
};
int main() {
    Base* bp; // Allowed pointer to abstract class
// Base obj; // x Error : can't create obj
    Derived d;
    bp = &d; // Allowed
    bp -> show(); // Output : Derived Class

Assignment - 22

Runtime Polymorphism:
Ability of a function to be dynamically bound at